M13 - Delta Lakehouse

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Concept before command

Every lesson explains the mental model and correctness/performance evidence before asking you to run the project.

Contents

DE13-01 - Data Lake vs Warehouse vs Lakehouse

What you will be able to do

Place Delta Lake in the lake/warehouse/lakehouse landscape and identify which problems table formats solve.

Why this matters

You already know Parquet; M13 adds reliable table operations/version history over file storage.

Picture it this way

Lakehouse keeps the flexible file yard but adds an official table ledger so writes/readers agree on consistent versions.

Start from zero

- Data lake is file/object-storage-oriented and flexible.
- Warehouse is managed relational analytics with strong table/query management.
- Lakehouse combines lake storage/open-ish files with a table protocol/metadata layer.
- Delta Lake is one table-format implementation using Parquet data + transaction log.
- A lakehouse still needs compute engine, catalog/governance, pipeline quality and access controls.
- Use M11 as storage foundation; M13 focuses on table transactions/operations.

Teacher demonstration

```
Parquet files + _delta_log + Delta protocol
        -> Delta table snapshots/transactions
Spark/other compatible engine
        -> reads/writes the table
```

Read the example like English

- The table format does not replace Spark; it coordinates table state on storage.
- The same Parquet bytes without the transaction log are just file data, not the same table semantics.

Expected check

layer	role
object/file storage	physical data
Delta log/protocol	table state/transactions
Spark/engine	compute/read/write
catalog/governance	discovery/access management

Common beginner traps

- Calling Delta a database server.
- Assuming lakehouse means no warehouse needed.
- Ignoring table protocol compatibility between engines.

Now you do a similar task

Inspect the supplied project and draw storage vs Delta log vs Spark responsibilities.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What does Delta add that Parquet file format alone does not provide?

Answer in your own words before continuing.

DE13-02 - Why plain files are not tables

What you will be able to do

Explain why a folder of files cannot safely support concurrent multi-file update/delete/current-table semantics without additional metadata/protocol.

Why this matters

Plain files have no single atomic table commit record; readers can list different file sets while a write is in progress.

Picture it this way

A pile of invoices has no official ledger saying which copies are current/cancelled; two people can disagree about the correct pile.

Start from zero

- A file write/rename may be atomic at object level but a table update can involve many files.
- A reader listing a directory during multi-file rewrite may see partial old/new sets.
- Deletes/updates require representing which old files are removed and which replacements are active.
- Schema/version/history need authoritative metadata.
- Delta transaction log records committed actions/version so readers choose a consistent snapshot.
- Do not manually delete/replace data files behind the table protocol.

Teacher demonstration

```
# Unsafe plain-file idea
write new_part_1.parquet
write new_part_2.parquet
# reader lists directory HERE -> partial state
remove old_part_*.parquet
# Delta instead commits file add/remove actions as one table version.
```

Read the example like English

- The logical table commit is distinct from individual file creation.
- Readers use committed log state rather than arbitrary directory timing.

Expected check

plain files challenge	Delta mechanism
partial multi-file update	atomic transaction version
current file set	log snapshot
delete/update	remove/add actions
history	version log

Common beginner traps

- Thinking object-storage listing equals a transaction.
- Editing Delta files outside protocol.
- Assuming one-file table has no need for metadata as it evolves.

Now you do a similar task

Write a failure timeline for a two-file plain-Parquet update and show what two readers might see; then explain how a committed table version changes that.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why is atomicity a table-level property rather than merely "each file write succeeded"?

Answer in your own words before continuing.

DE13-03 - Transaction log, ACID and table versions

What you will be able to do

Inspect Delta transaction history and connect ACID properties to concrete table versions/actions.

Why this matters

ACID terms are useful only if you can explain what they mean for a failed/concurrent file-table write.

Picture it this way

Every successful transaction adds a numbered ledger entry; readers select one committed ledger state rather than half-written changes.

Start from zero

- Atomicity: transaction changes become visible as committed version together or not.
- Consistency: table protocol/schema/constraints supported by engine keep valid table state, but business quality rules still yours.
- Isolation: concurrent operations coordinate through transaction protocol so committed readers see consistent snapshots.
- Durability: committed table state/log/data persists on storage.
- Each successful transaction creates a new version/history record.
- Inspect _delta_log/history; do not infer history from file modified times.

Teacher demonstration

```
python delta_runtime_probe.py
python M13_G02_LOG_HISTORY.py
# Inspect table history/version and _delta_log files.
```

Read the example like English

- Runtime probe tells whether Spark+Delta environment is available.
- History script shows table versions/operations.
- Version is a logical table commit sequence.

Expected check

ACID	concrete Delta idea
A	one committed version
C	valid protocol/schema state
I	consistent snapshot/concurrency handling
D	committed log/data persist

Common beginner traps

- Using ACID as marketing word with no failure example.
- Assuming business duplicates are prevented by ACID.
- Manual file operations outside transaction log.

Now you do a similar task

Create a tiny table and perform two commits; inspect history and explain which version each read represents.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Which ACID property prevents readers from treating half a multi-file update as the completed table state?

Answer in your own words before continuing.

DE13-04 - Create, read, update and delete

What you will be able to do

Perform Delta create/read/update/delete while validating versions and row/key totals after each mutation.

Why this matters

File-based update/delete rewrites underlying data files. Delta provides table operations, but you still need correct predicates and controls.

Picture it this way

Update/delete is editing the official ledger-backed table, not opening Parquet files in-place.

Start from zero

- Create/write modes must be explicit (error/append/overwrite) and safe for target path.
- UPDATE predicate determines affected rows; validate count/keys before/after.
- DELETE removes matching logical rows in new table version, not necessarily immediately deleting old physical files.
- Every mutation creates transaction history and may leave old files for time travel until cleanup.
- Read current version through Delta API/format, not arbitrary Parquet directory scan.
- Use disposable training paths for destructive delete/update practice.

Teacher demonstration

```
python M13_G01_CREATE_READ.py
python M13_G03_UPDATE_DELETE.py
# Save history + row/key controls after each operation.
```

Read the example like English

- The scripts create a training Delta table then mutate it.
- History should show operations/versions.
- Row/control changes must match predicates exactly.

Expected check

operation	validation
create	expected initial rows
update	only intended keys changed
delete	only intended keys removed
history	new version per commit

Common beginner traps

- UPDATE without first checking predicate population.
- Reading underlying Parquet files directly and calling it table truth.
- Vacuuming old files immediately after delete.

Now you do a similar task

Run create/read and update/delete labs. Before each mutation, query the intended keys/count; after, validate exact changes and history.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why can deleted data still exist in old physical files after a Delta DELETE?

Answer in your own words before continuing.

DE13-05 - Schema enforcement and schema evolution

What you will be able to do

Use schema enforcement as a protection and controlled evolution as a reviewed contract change.

Why this matters

Delta can reject incompatible writes, preventing accidental table corruption; evolution convenience must remain intentional.

Picture it this way

The table has an approved form template. A new optional field may be approved; a producer cannot silently replace quantity integer with random text.

Start from zero

- Schema enforcement checks writes against current table schema.
- Append with mismatched columns/types can fail instead of silently reshaping table.
- Controlled schema evolution options such as mergeSchema can add compatible columns when deliberately enabled.
- Type changes/renames/drops need explicit migration/consumer compatibility plan.
- Schema changes create new table metadata/version; document/test downstream readers.
- Do not globally enable permissive evolution to make every source drift pass.

Teacher demonstration

```
python M13_G05_SCHEMA.py
# First observe enforcement behavior, then controlled evolution path.
```

Read the example like English

- The lab should include one rejected mismatch and one approved additive evolution.
- Record table schema before/after and history.

Expected check

case	expected
compatible original write	success
unsafe mismatch	fail
approved additive field	evolve intentionally
downstream	compatibility re-tested

Common beginner traps

- Treating mergeSchema as a data-cleaning option.
- Changing target table whenever source sends unexpected fields.
- No schema-version lineage.

Now you do a similar task

Run schema lab and write a producer/consumer contract note explaining why the accepted evolution is compatible and rejected case is not.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What risk appears if every ingestion job enables schema evolution automatically?

Answer in your own words before continuing.

DE13-06 - Time travel and version history

What you will be able to do

Read earlier table versions and use history for debugging/recovery while respecting retention.

Why this matters

When a bad update lands, time travel lets you inspect before/after state without restoring an entire backup first.

Picture it this way

History is a stack of saved ledger snapshots; time travel opens an earlier snapshot for comparison.

Start from zero

- Delta history lists version, timestamp, operation and operation metrics depending runtime.
- Read using versionAsOf/timestampAsOf where supported.
- Time travel is useful for audit/debug/reproduce old model input.
- It is not a permanent backup: VACUUM/retention can remove old data files.
- Do not overwrite current table just to inspect old state.
- Recovery may use restore/rewrite based on an earlier version, but validate and document the new commit.

Teacher demonstration

```
python M13_G04_TIME_TRAVEL.py
# Compare current version with versionAsOf=<earlier_version>.
```

Read the example like English

- The earlier read should show pre-mutation rows/values.
- Current table remains unchanged by reading history.
- Record exact version IDs in evidence.

Expected check

use	proof
debug bad update	compare versions
reproduce historical input	read exact version
backup replacement	NO
retention dependency	old files must still exist

Common beginner traps

- Calling time travel a backup strategy.
- Using timestamp without timezone/commit clarity.
- VACUUM aggressively before investigation period expires.

Now you do a similar task

Perform an update then compare current vs prior version. Identify the exact key/value difference and version numbers.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why is versionAsOf more reproducible than "read what table looked like yesterday" without a version?

Answer in your own words before continuing.

DE13-07 - MERGE, upserts and idempotent incremental loads

What you will be able to do

Design Delta MERGE for idempotent incremental insert/update with correct business key and source winner rules.

Why this matters

Incremental batches often repeat keys/replay. A MERGE can make current-state application safe, but only if match and version logic are correct.

Picture it this way

Use the business ID to find the existing customer folder; update only if incoming card is newer, insert if missing.

Start from zero

- Match on stable business key.
- Pre-deduplicate source to one intended record per key.
- Use updated_at/source_version condition so stale records do not overwrite newer target state.
- whenNotMatched inserts new keys; whenMatched updates selected columns.
- Same batch replay should leave target state/key count unchanged.
- Validate Delta history + source/target controls after merge.

Teacher demonstration

```
python M13_G06_MERGE.py
# Inspect the source updates, match condition and target before/after/replay.
```

Read the example like English

- The supplied script demonstrates update + insert.
- Your reasoning must include duplicate source keys and stale update policy even if tiny sample is clean.

Expected check

test	expected
new ID	insert
existing newer ID	update
stale update	ignored by version rule if implemented
exact replay	stable state

Common beginner traps

- MERGE condition using mutable field.
- Source duplicate keys unresolved.
- Replay never tested.

Now you do a similar task

Run MERGE twice with the same update batch and prove target business-key count/values stay stable. Add a stale update scenario and define expected behavior.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What makes MERGE idempotent: the command name, or your key/version conditions?

Answer in your own words before continuing.

DE13-08 - Slowly Changing Dimension Type 2 with Delta

What you will be able to do

Implement SCD Type 2 in Delta with closing old versions and inserting new versions while preserving one current row per business key.

Why this matters

Current-state MERGE loses attribute history; SCD2 keeps historical dimension versions for time-aware analytics.

Picture it this way

Do not erase the old address card: date-close it and add a new current card with a new surrogate/history row.

Start from zero

- SCD2 row has business key, attributes, effective_from/effective_to and is_current (often surrogate key).
- Changed current row is expired; new version inserted.
- Unchanged source should not create a new version.
- Exact replay of same change should not create another duplicate history row.
- Validate one-current-row invariant and non-overlapping ranges.
- Late/out-of-order effective dates require a more advanced policy; do not assume simple current-update handles every history correction.

Teacher demonstration

```
python M13_G07_SCD2.py
# Inspect rows for changed customer before/after and rerun behavior.
```

Read the example like English

- The lab is a simplified SCD2 training implementation.
- Your controls should count total versions and current versions per business key.

Expected check

invariant	proof
one current per key	group/count is_current
old version	effective_to set
new version	effective_from + is_current true
replay	no extra duplicate version

Common beginner traps

- Updating current row in place.
- Creating new version without closing old.
- Ignoring replay duplicates.

Now you do a similar task

Run SCD2 lab, select the changed business key history ordered by effective_from, and explain every version. Rerun the change.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why does a surrogate/history row identity help when one business key has several SCD2 versions?

Answer in your own words before continuing.

DE13-09 - Bronze, Silver and Gold layer contracts

What you will be able to do

Build Bronze/Silver/Gold as explicit contracts and decide which transformations/quality guarantees belong at each layer.

Why this matters

Medallion architecture is only useful if layers have meaning, not if every pipeline copies the same data three times.

Picture it this way

Bronze is receiving evidence, Silver is inspected standardized material, Gold is consumer-ready product.

Start from zero

- Bronze preserves source-like data + ingestion metadata and supports replay.
- Silver applies parsing, dedup, schema/quality rules and conforming at declared reusable grain.
- Gold shapes dimensions/facts/aggregates for specific consumers/SLAs.
- Bad rows may be quarantined alongside Bronze/Silver with reasons; do not silently drop.
- Each layer must define key/grain/schema/freshness and rebuild source.
- Delta transactions help reliable tables within each layer; they do not define the layer business contract.

Teacher demonstration

```
python M13_G08_MEDALLION.py
# Inspect Bronze -> Silver -> Gold row/key/control transitions.
```

Read the example like English

- The script provides a concrete small medallion flow.
- Save counts and what changed at each boundary.
- A Gold aggregate should reconcile to Silver population/measure.

Expected check

layer	contract
Bronze	source-like/replayable
Silver	validated/conformed reusable
Gold	consumer-ready modeled/aggregated

Common beginner traps

- Cleaning/deduping Bronze until raw evidence disappears.
- Gold as just another copy of Silver.
- No reconciliation between layers.

Now you do a similar task

Run medallion lab and write a contract for each table: grain, key, quality and rebuild source.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

If Bronze is supposed to support replay, which transformations are dangerous to apply there?

Answer in your own words before continuing.

DE13-10 - File layout, compaction and table maintenance

What you will be able to do

Reason about Delta file layout, partitioning, compaction/OPTIMIZE-like maintenance and VACUUM/retention safely.

Why this matters

Table transactions solve consistency, not automatically efficient physical layout. Many small files/poor partitions still hurt reads.

Picture it this way

The ledger can be perfect while the warehouse floor has millions of tiny boxes; maintenance reorganizes boxes without changing inventory truth.

Start from zero

- Delta writes create Parquet data files; partitioning determines directory grouping.
- Repeated small writes/MERGE can accumulate small files.
- Compaction rewrites into fewer larger files while transaction log updates active file set.
- Managed platforms may provide OPTIMIZE/ZORDER-style features; open/local runtimes differ, so know what your engine supports.
- VACUUM physically removes unreferenced old files after retention rules; can reduce time-travel window.
- Validate row/key/measure totals after layout maintenance.

Teacher demonstration

```
python M13_G09_LAYOUT.py
# Inspect part-file counts/partitions before/after the training rewrite.
# Do NOT run aggressive VACUUM outside a disposable lab.
```

Read the example like English

- The local lab focuses on file-count/repartition reasoning rather than cloud-specific OPTIMIZE commands.
- History remains logical truth while physical files change across versions.

Expected check

maintenance	goal/risk
compaction	fewer files; preserve data
partitioning	pruning; avoid high cardinality
VACUUM	remove old unreferenced files; reduces recovery/time travel

Common beginner traps

- Running VACUUM with unsafe short retention copied from a tutorial.
- Partitioning by unique ID.
- Declaring compaction successful without row totals.

Now you do a similar task

Inspect layout lab file counts and propose a production partition/compaction/retention policy for daily order tables.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why are compaction and VACUUM different operations?

Answer in your own words before continuing.

DE13-11 - Lakehouse integration and debugging

What you will be able to do

Debug an end-to-end lakehouse job by separating Spark/runtime, Delta protocol, schema, merge, layout and data-quality failures.

Why this matters

Lakehouse failures can occur at several layers. Random edits to Spark/Delta versions or table files can make recovery worse.

Picture it this way

Diagnose the layer like a mechanic: engine starts? table log valid? schema compatible? merge key correct? data controls balanced?

Start from zero

- Start with runtime probe/version compatibility when session/table-format fails.
- Inspect table history/log before editing/deleting files.
- Classify schema enforcement error vs data-quality/business rejection.
- For MERGE/SCD2, validate source key uniqueness/version ordering first.
- For performance, inspect file count/partition/plan after correctness.
- Keep raw/previous table versions for replay; do not repair by manually deleting files.
- Final practical should demonstrate replay and controlled failure recovery.

Teacher demonstration

```
# Debug order
1. delta_runtime_probe.py
2. table path + _delta_log/history
3. schema/current version
4. source key/version quality
5. operation error/log
6. row/key/measure reconciliation
7. layout/performance evidence
```

Read the example like English

- The order narrows causes from runtime to protocol to data/model.
- Manual file deletion is intentionally absent from the repair list.

Expected check

symptom	first check
Delta session import/runtime error	versions/runtime probe
schema mismatch	table/source schemas
duplicate MERGE target effect	source key uniqueness + match rule
slow reads	layout/plan after correctness

Common beginner traps

- Reinstalling Spark/Delta immediately.
- Deleting _delta_log.
- Performance tuning before reconciliation passes.

Now you do a similar task

Complete the module practical/fresh failure pack, write a diagnosis timeline and save evidence before/after repair.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why should you inspect table history before deleting any suspicious data file?

Answer in your own words before continuing.